EcoHealth Dengue Outbreak Predictor

Subtitle: A Climate-Integrated Disease Surveillance System for Sri Lanka

---

I. Executive Summary

Dengue fever remains a major public health challenge in Sri Lanka, with seasonal outbreaks closely linked to environmental conditions such as rainfall, temperature, and humidity. Driven by climate variability and rapid urbanization, monsoon-driven outbreaks frequently lead to hospital overcrowding, emergency resource shortages, and delayed vector-control interventions.

EcoHealth proposes a predictive, web-based surveillance system that integrates environmental weather patterns with clinical hospital case records. By shifting dengue control from reactive responses to proactive, risk-based planning, the system generates early outbreak risk scores. This empowers health authorities with actionable recommendations to support early interventions, reduce outbreak severity, and contribute to sustainable public health management.

II. Business Case & Problem Definition

The Problem

Current dengue monitoring systems in Sri Lanka are predominantly reactive, focusing on recording cases *after* infections occur rather than predicting high-risk periods in advance. This creates several operational pain points:

- **Process Inefficiencies:** Manual or semi-manual tracking of dengue cases at the district level with no automated integration of local climate data.
- **Financial Strain:** Increased emergency treatment and overtime costs during unanticipated spikes, alongside inefficient deployment of fogging operations.
- **Product Gaps:** Existing dashboards lack predictive analytics, focusing entirely on historical case counts without simple, web-based district-level risk scoring.

Research Gap & Evidence

The Ministry of Health reports tens of thousands of dengue cases annually, heavily peaking post-monsoon in districts like Colombo, Gampaha, and Kandy. International studies confirm that climate variables strongly influence dengue transmission dynamics with a time lag of several weeks.

- **The Gap:** There is no widely deployed district-level early warning system offering a 2–4 week prediction window in Sri Lanka.
- **Field Insight:** Informal discussions with field health officers indicate that *"fogging usually starts only after a large number of cases are reported."*

Stakeholder Analysis

Stakeholder	Primary Problem Faced	Expected Benefit from EcoHealth
Public Health Officers	Late outbreak detection	Early warnings and prioritized hotspots
Hospitals	Overcrowding and resource strain	Better capacity and resource planning
Govt. / Ministry	Limited data-driven planning	Evidence-based policy and budgeting
General Public	High infection risk and low	Timely alerts and prevention guidance

III. Proposed Solution & Core Logic

The EcoHealth Platform

The proposed solution is a Django-based predictive web application that functions as a climate-integrated dengue early warning system. It features a central **Risk Calculation Engine** that aggregates 14–28 days of time-lagged environmental data, current clinical case counts, and hospital capacity to output a unified Dengue Risk Score (0–100).

Crucially, to maintain data integrity, the system features a **Secure Authentication Layer** ensuring that clinical data can only be entered and modified by verified personnel tied to specific geographic districts.

System Logic & Risk Model

Dengue incidence does not respond instantly to weather changes. To account for mosquito breeding cycles and viral incubation, the system uses a **Time-Lagged Climate–Disease Relationship**.

The Risk Score Formula:

Total Risk Score = Environmental Score + Clinical Score + Capacity Score

- **Environmental Score (0–60):** Assesses the last 2–4 weeks of temperature, rainfall, and humidity.

    - *Example:* Optimal mosquito breeding temperatures (28–30°C) and heavy multi-day rainfall trigger maximum sub-scores.
- **Clinical Score (0–30):** Measures active virus circulation based on current district case counts.
    - *Example:* >50 active cases trigger a maximum clinical score.
- **Capacity Score (0–20):** Measures the vulnerability of the healthcare system.
    - *Example:* Hospital bed usage >80% triggers maximum vulnerability scores.

Risk Levels & Interpretation

Score Range	Risk Level	Recommended Action
0–30	**Low**	Routine monitoring is sufficient.
31–60	**Medium**	Increased surveillance and community awareness.
61–80	**High**	Step up vector-control operations and hospital prep.
81–100	**Critical**	Immediate, intensive response and resource mobilization.

IV. Technical Architecture & Implementation

Technology Stack

Component	Technology
Backend & Domain Logic	Python, Django
API Layer	Django REST Framework
Database	SQLite (Django ORM)
Frontend UI	Django Templates, Tailwind CSS
Data Visualization	Chart.js
Cloud Deployment	Render

System Architecture

The system follows a three-layer modular architecture:

1. **Interface Layer:** District risk dashboard, charts, secure data entry forms, and role-based login pages (Tailwind CSS/Chart.js).

2. **Domain Layer:** The RiskCalculationEngine governing time-lag aggregation, scoring logic, and business rules for alerts.
3. **Persistence Layer:** Django ORM models for environmental, clinical, and risk data, including user audit trails.

Database Schema (High-Level)

To support secure data entry and maintain a reliable audit trail, the database includes role-based profiles linked to Django's built-in authentication:

- **User** *(Django Built-in)*: id, username, password
- **UserProfile**: id, user_id (FK), district_id (FK), role *(e.g., Data Clerk, Health Officer)*
- **District**: id, name
- **EnvironmentalLogs**: id, district_id, date, temperature, rainfall, humidity
- **HospitalAdmissions**: id, district_id, date, active_cases, available_beds, **entered_by** *(FK to User for audit trail)*
- **RiskScores**: id, district_id, date, risk_score, risk_level

V. Project Delivery & Quality Assurance

Agile Implementation Plan (4-Week Sprint)

- **Week 1: Initiation (Research & Design)** - Finalize problem statement, stakeholder analysis, ERDs, architecture diagrams, and engine pseudocode.
- **Week 2: Backend Development** - Implement Django project, core models (including User authentication), RiskCalculationEngine, unit tests, and Weather API integration.
- **Week 3: UI, API & Testing** - Build dashboards, role-based login portals, REST endpoints, and perform functional/edge testing. Deploy initial build to Render.
- **Week 4: Finalization** - UI polish, error handling refinement, documentation (README, technical reports), and live deployment stabilization.

Team Composition

- **Project Manager / Lead Developer:** G. B. Darsha Anuradha
- **Domain Researcher:** Name
- **Backend Developer:** Name
- **API Developer:** Name
- **Frontend Developer:** Name
- **QA & DevOps:** Name

Testing Strategy

- **Unit Tests:** Validation of the RiskCalculationEngine using synthetic "Low Risk" and "Critical Risk" scenarios.
- **Integration Tests:** End-to-end data flow validation (Input → Database Audit Trail → Calculation → Display).
- **Security & Edge Case Testing:** Unauthorized access attempts, API failure handling, and incomplete hospital data fallbacks.

VI. Outcomes & Future Roadmap

Expected Outcomes

1. A fully functional, cloud-deployed web application accessible via a public URL.
2. A functional early-warning tool demonstrating an immediate improvement in resource planning for hospital administrators and field officers.
3. An integrated "Eco-Health Dashboard" feature estimating potential cases avoided through optimized, early-intervention fogging.

Future Work

- **Mobile Alerts:** SMS-based or push-notification alerting for field officers.
- **GovTech Integration:** Direct API linking with official government open-data platforms.
- **Scalability:** Expanding the logic to cover other climate-influenced vector-borne diseases